

Práctica 03

Nombre del Equipo: Las pros y Diego.

Práctica: 03

Integrantes:

- Ciros Ortiz Diego - No. Cuenta: 323238300
- Soto Álvarez Regina - No. Cuenta: 323313117
- Santuario Téllez Melanie Elizabeth - No. Cuenta: 323136385

Descripción de la práctica.

Se implementó una simulación para la organización de actividades en la "*Academia Ninja de la Aldea de las Ciencias*". El sistema permite:

- ★ Registro de aspirantes: Almacena la información de los prospectos o aspirantes (nombre, edad, clan y nivel de habilidad) en una HashTable.
- ★ Registro de ninjas voluntarios: Almacena la información de los ninjas disponibles (nombre, edad, clan, rango y nivel de habilidad) en un Array.
- ★ Formación de grupos: Asigna un ninja voluntario como líder y una cantidad máxima de aspirantes dependiendo de su rango (Genin: 1, Chunin: 2, Jonin: 3).
- ★ Asignación de paquetes de herramientas: Utiliza paquetes prefabricados (Básico, Avanzado, Táctico) o crea uno personalizado con un peso total calculado.
- ★ Asignación de campo de entrenamiento: Determina el campo (Valle del Dragón, Bosque Sombrío, Montaña Espiritual) según la suma de niveles de habilidad de los integrantes del grupo.
- ★ Resumen final: Muestra un reporte con los integrantes de cada grupo, el paquete adquirido con su peso total y el campo asignado con su descripción.

Patrones aplicados.

- ★ *Iterator*: Se eligió el patrón Iterator para poder recorrer de manera uniforme las colecciones de aspirantes (almacenados en una HashTable) y de ninjas voluntarios (almacenados en un Array). Como ambos se guardan en estructuras de datos diferentes, el patrón Iterator nos permite recorrerlos con una interfaz común (hasNext() y next()) sin que el cliente (la clase AcademiaNinjas) tenga que conocer los detalles internos de cada estructura. Esto facilita la formación de grupos y el acceso a los datos.
- ★ *Builder*: Se aplicó el patrón Builder para la construcción de los paquetes de herramientas. El problema requiere crear paquetes prefabricados o ya hechos (Básico, Avanzado, Táctico) y también paquetes personalizados. El patrón Builder permite construir estos objetos paso a paso de esta manera podemos tener diferentes "constructores" (ConstructorPaqueteConcreto, ConstructorPaqueteAvanzado, etc.) que siguen el mismo proceso pero que cada uno genera un paquete distinto, y permite crear un paquete personalizado sin la necesidad de crear una clase grande con múltiples constructores.

- ★ *Factory*: Se utilizó el patrón Factory para la creación de los campos de entrenamiento. La lógica del problema nos dice que el campo depende de la suma de los niveles de habilidad del grupo y que esta condición podría cambiar o ampliarse en el futuro. Con el patrón Factory, centramos la creación de los objetos (ValleDelDragon, BosqueSombrio, MontanaEspiritual) en una clase creadora (CreadorCampoConcreto), lo que permite que el cliente no tenga que conocer las clases concretas ni las condiciones para instanciarlas. Si en el futuro se agrega un nuevo campo, solo se modifica la fábrica.

Versión de Java.

- ★ Versión de Java utilizada: JDK 23

Instrucciones de compilación y ejecución.

Para compilar (desde la carpeta src/):

```
rm -rf bin && mkdir bin && javac -d bin *.java
```

En Windows (CMD / PowerShell desde la carpeta src/):

Compilar (CMD):

```
if exist bin rmdir /s /q bin && mkdir bin && javac -d bin *.java
```

Compilar (PowerShell):

```
if (Test-Path bin) { Remove-Item -Recurse -Force bin }; New-Item -ItemType Directory -Name bin; javac -d bin *.java
```

Para ejecutar (desde la carpeta src/):

```
java -cp bin Main
```

Observaciones adicionales.

- ★ Los diagramas UML se encuentran en la carpeta raíz Practica03/ en formato PNG.